

Assignment Q4

Deploying a WAR to a second EC2 instance (Tomcat) from Jenkins

Jenkins (JM) builds the WAR from the repo. The deployment target is a separate EC2 instance running Tomcat, not a Jenkins slave.

Prerequisite: Jenkins Server Setup

Do this on the Jenkins EC2 instance (Amazon Linux 2023) before starting the Q4 steps below.

Setup Step 1: Install Java

```
sudo yum install java-21-amazon-corretto.x86_64 -y
java -version
```

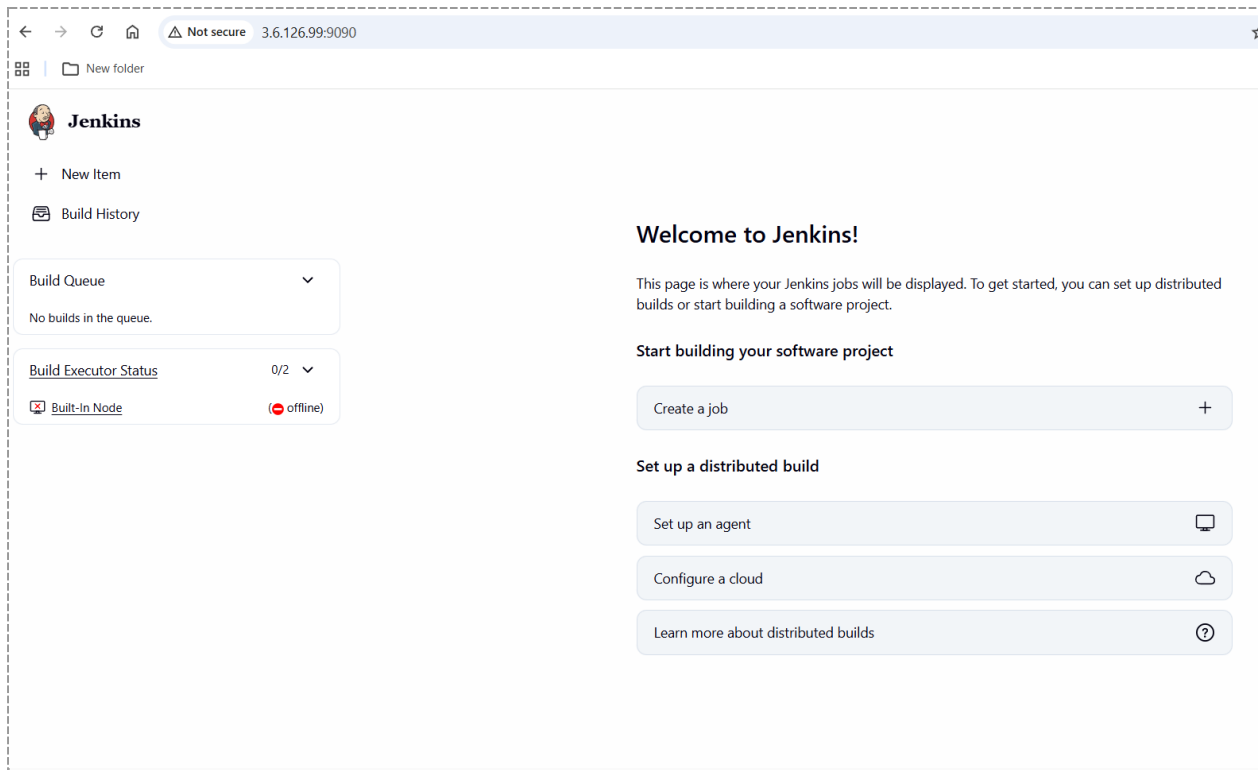
Setup Step 2: Install and start Jenkins

```
sudo wget -O /etc/yum.repos.d/jenkins.repo https://pkg.jenkins.io/redhat-stable/jenkins.repo
sudo rpm --import https://pkg.jenkins.io/redhat-stable/jenkins.io-2023.key
sudo yum install jenkins -y
sudo systemctl enable jenkins
sudo systemctl start jenkins
sudo systemctl status jenkins
```

Note: if Tomcat is already running on port 8080 on this same server, Jenkins will fail to start (port conflict). Move Jenkins to another port, for example 9090:

```
sudo mkdir -p /etc/systemd/system/jenkins.service.d
printf '[Service]\nEnvironment="JENKINS_PORT=9090"\n' | sudo tee
/etc/systemd/system/jenkins.service.d/override.conf
sudo systemctl daemon-reload
sudo systemctl reset-failed jenkins
sudo systemctl start jenkins
```

Also open the Jenkins port (8080, or 9090 if changed) in this instance's security group.



Setup Step 3: Unlock Jenkins and install plugins

- Open `http://<JENKINS-EC2-PUBLIC-IP>:8080` (or `:9090` if you changed the port).
- Get the initial admin password:

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

- Choose Install suggested plugins, then create the admin user.
- Install the SSH Agent plugin: Manage Jenkins → Plugins → Available plugins → "SSH Agent" (needed for the sshagent deploy step in Q4).



Setup Step 4: Install and verify Git

Git must be available on the Jenkins VM so that Jenkins can check out the GitHub repository.

```
sudo yum install git -y
```

```
git --version
```

```
2026-10-07 16:53.31 /home/mobaxterm/Desktop/aws key git --version  
git version 2.45.1
```

Setup Step 5: Install Maven

Needed to build the WAR (target/LoginWebApp.war) if the project is a Maven project.

```
sudo yum install maven -y  
mvn -version
```

```
2026-10-07 17:00.09 /home/mobaxterm/Desktop/aws key mvn -version  
MINGW support requires --add-opens java.base/java.lang=ALL-UNNAMED  
MINGW support requires --add-opens java.base/java.lang=ALL-UNNAMED  
Apache Maven 3.9.16 (2bdd9fddda4b155ebf8000e807eb73fd829a51d5)  
Maven home: C:\Tools\apache-maven-3.9.16  
Java version: 25.0.3, vendor: Eclipse Adoptium, runtime: C:\Users\darsh\AppData\Local\Programs\Eclipse Adoptium\jdk-25.0.3.9-hotspot  
Default locale: en_IN, platform encoding: UTF-8  
OS name: "windows 11", version: "10.0", arch: "amd64", family: "windows"
```

Setup Step 6: Create the Jenkins Freestyle project

- On the Jenkins dashboard, click New Item.
- Enter a name, for example LoginWebApp-Freestyle-Deployment.
- Select Freestyle project and click OK.

Jenkins / New Item

New Item

Enter an item name

LoginWebApp-Freestyle-Deployment

Select an item type

- ☐ Pipeline
Build, test, and deploy using pipelines. Supports stages, parallel work, and running on multiple agents.
- ☒ Freestyle project
Classic, general-purpose job type that checks out from up to one SCM, executes build steps serially, followed by post-build steps like archiving artifacts and sending email notifications.
- ☐ Multi-configuration project
Suitable for projects that need a large number of different configurations, such as testing on multiple environments, platform-specific builds, etc.
- ☐ Folder
Creates a container that stores nested items in it. Useful for grouping things together. Unlike view, which is just a filter, a folder creates a separate namespace, so you can have multiple things of the same name as long as they are in different folders.
- ☐ Multibranch Pipeline
Creates a set of Pipeline projects according to detected branches in one SCM repository.

OK

Setup Step 7: Configure the Git repository (Source Code Management)

Open the job → Configure, then:

- Source Code Management → select Git.
- Repository URL: <https://github.com/Shantanumajan6/project>
- Credentials: none (public repository).
- Branch Specifier: */master

Note: a manual git clone is not required. Jenkins performs the Git checkout into its workspace before running the build steps.



The screenshot shows the Jenkins configuration interface for a Git repository. The 'Repository URL' is set to `https://github.com/Shantanumajan6/project.git`. The 'Credentials' are set to 'none'. The 'Branches to build' section shows a 'Branch Specifier (blank for \'any\')' set to `*/master`. There are buttons for 'Add Repository' and 'Add'.

Setup Step 8: Add the build step and run the job

Build Steps → Add build step → Execute shell:

```
#!/bin/bash
set -e

echo "==== Jenkins Workspace ====="
pwd
ls -la

echo "==== Adding compatible WAR plugin temporarily ====="

sed -i '/<finalName>LoginWebApp<\/finalName>\/a\
    <plugins>\
      <plugin>\
        <groupId>org.apache.maven.plugins<\/groupId>\
        <artifactId>maven-war-plugin<\/artifactId>\
        <version>3.4.0<\/version>\
      <\/plugin>\
    <\/plugins>' pom.xml

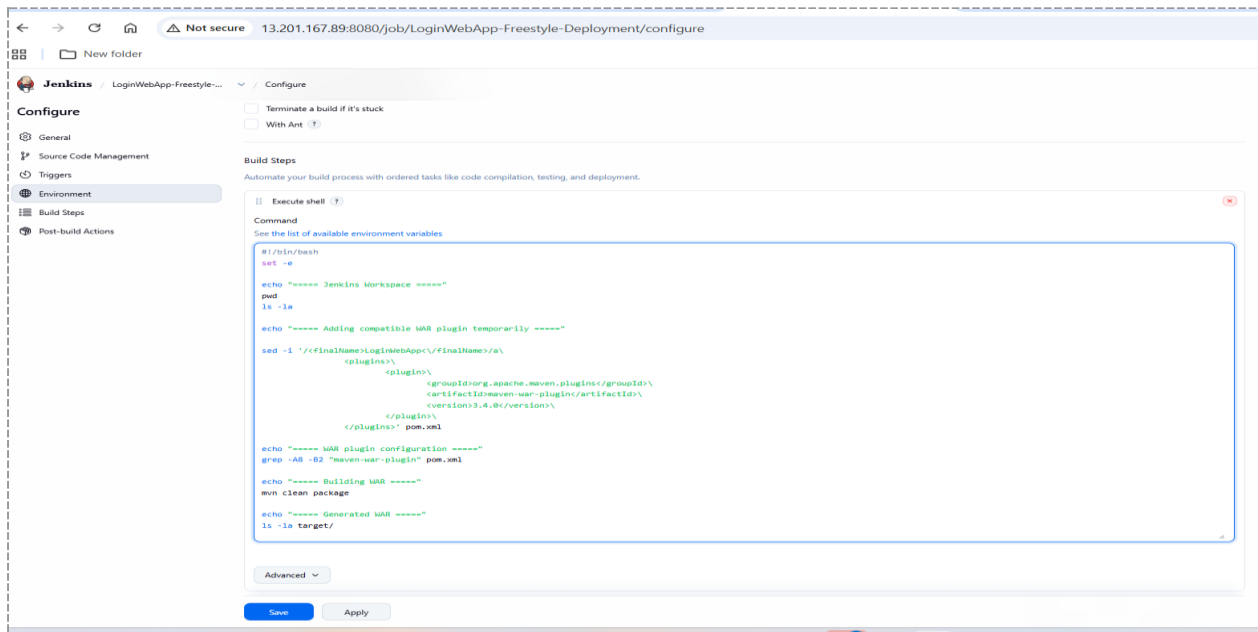
echo "==== WAR plugin configuration ====="
grep -A8 -B2 "maven-war-plugin" pom.xml
```

```
echo "==== Building WAR ====="
mvn clean package

echo "==== Generated WAR ====="
ls -la target/
```

- Click Save, then Build Now.
- Open the build number → Console Output and confirm it ends with Finished: SUCCESS.

The deploy to the second EC2 is added later in Q4, Step 5.



Q4: Use a Second EC2 Instance as the Deployment Target

Step 1: Launch a new EC2 instance

- Go to AWS Console → EC2 → Launch Instance.
- Choose Amazon Linux 2023 (or Ubuntu if you prefer).
- Instance type: t2.micro (enough for testing).
- Key pair: use the same .pem key you already have.
- Security group: allow SSH (22) and the Tomcat port (8081, or whichever you set) inbound.

Instances (2) Info									
Saved filter sets Choose filter set Filter Instance by attribute or tag (case-sensitive) Instance state: running Clear filters									
Name	Instance ID	Instance state	Instance type	Status check	Availability Zone	Public IPv4 DNS	Public IPv4 ...	Elastic IP	IPv6 IPs
ec2-jen2	i-016cf7c397eae75d0	Running	t3.micro	3/3 checks pass	ap-south-1b	ec2-15-252-140-193.ap...	15.252.140.193	-	-
jenkins-server	i-0fffd76b4d2be005c9	Running	t3.micro	3/3 checks pass	ap-south-1b	ec2-13-201-167-89.ap...	13.201.167.89	-	-

Step 2: Install Java

On the new EC2 instance:

```
sudo yum install java-21-amazon-corretto.x86_64 -y
```

Step 3: Install Tomcat

Download and extract Tomcat:

```
wget https://dlcdn.apache.org/tomcat/tomcat-10/v10.1.60/bin/apache-tomcat-10.1.60.tar.gz
sudo mkdir -p /opt/tomcat
sudo tar xzvf apache-tomcat-10.1.60.tar.gz -C /opt/tomcat --strip-components=1
sudo chmod +x /opt/tomcat/bin/*.sh
sudo chown -R ec2-user:ec2-user /opt/tomcat
```

The chown line lets ec2-user copy the WAR into /opt/tomcat/webapps when Jenkins deploys.

Start Tomcat:

```
/opt/tomcat/bin/startup.sh
```

```
[root@ip-172-31-8-29 ~]# /opt/tomcat/bin/startup.sh
Using CATALINA_BASE:   /opt/tomcat
Using CATALINA_HOME:   /opt/tomcat
Using CATALINA_TMPDIR: /opt/tomcat/temp
Using JRE_HOME:        /usr
Using CLASSPATH:        /opt/tomcat/bin/bootstrap.jar:/opt/tomcat/bin/tomcat-juli.jar
Using CATALINA_OPTS:
Tomcat started.
[root@ip-172-31-8-29 ~]#
```

Step 4: Verify Tomcat

Check from the instance:

```
curl -I http://localhost:8080/
```

Or from a browser:

```
http://<NEW-EC2-PUBLIC-IP>:8080/
```

```
[root@ip-172-31-8-29 ~]# curl -I http://localhost:8080/
HTTP/1.1 200
Content-Type: text/html; charset=UTF-8
Date: Wed, 07 Oct 2026 12:22:15 GMT

[root@ip-172-31-8-29 ~]#
```

Step 5: Configure Jenkins to deploy remotely

On your Jenkins EC2:

1. Add your .pem key: Manage Jenkins → Credentials → System → Global credentials → Add Credentials. Kind: SSH Username with private key, ID: ec2-key, Username: ec2-user, paste the contents of the .pem file.
2. Add the deploy step using Option A (Freestyle, as in Assignment 3) or Option B (Pipeline).

Option A: Freestyle project

- Open the job → Configure → Build Environment → tick SSH Agent and select the ec2-key credential.
- Build Steps → Execute shell (replace the earlier build step with this one):

```
#!/bin/bash
mvn clean package
scp -o StrictHostKeyChecking=no target/LoginWebApp.war ec2-user@<NEW-EC2-IP>:/opt/tomcat/webapps/
ssh -o StrictHostKeyChecking=no ec2-user@<NEW-EC2-IP> "/opt/tomcat/bin/shutdown.sh || true; /opt/tomcat/bin/startup.sh"
```

Step 6: Test deployment

After the pipeline runs, open:

```
http://15.252.140.193:8080/LoginWebApp/
```

Not secure 15.252.140.193:8080/LoginWebApp/

New folder

Login Page

Username

Password

New User [Register Here](#)

Summary

You do not need a Jenkins slave. Launch a second EC2 instance, install Java and Tomcat, and let Jenkins deploy the WAR remotely via SSH/SCP using your existing .pem key.